Jonathan’s User Story 1: Verified Profile Creation

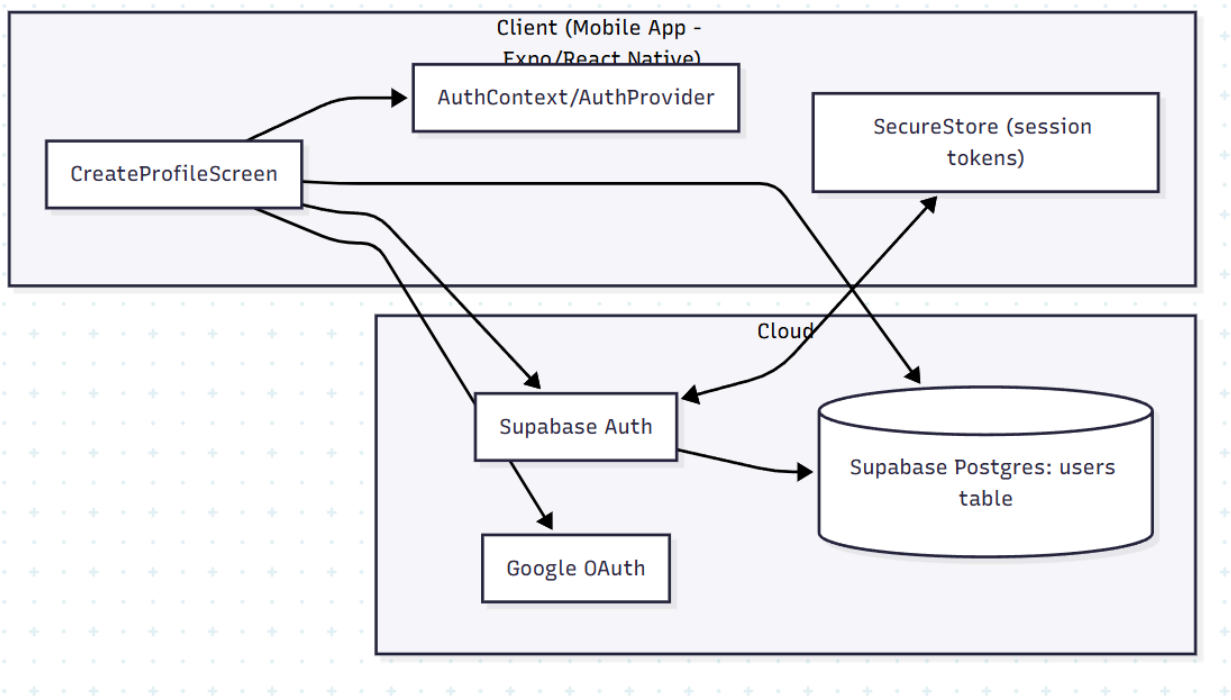
User Story:
As a buyer or seller, I want to create a verified profile so that I can establish my identity and build trust with my local community.

Primary Owner: Jonathan Gu

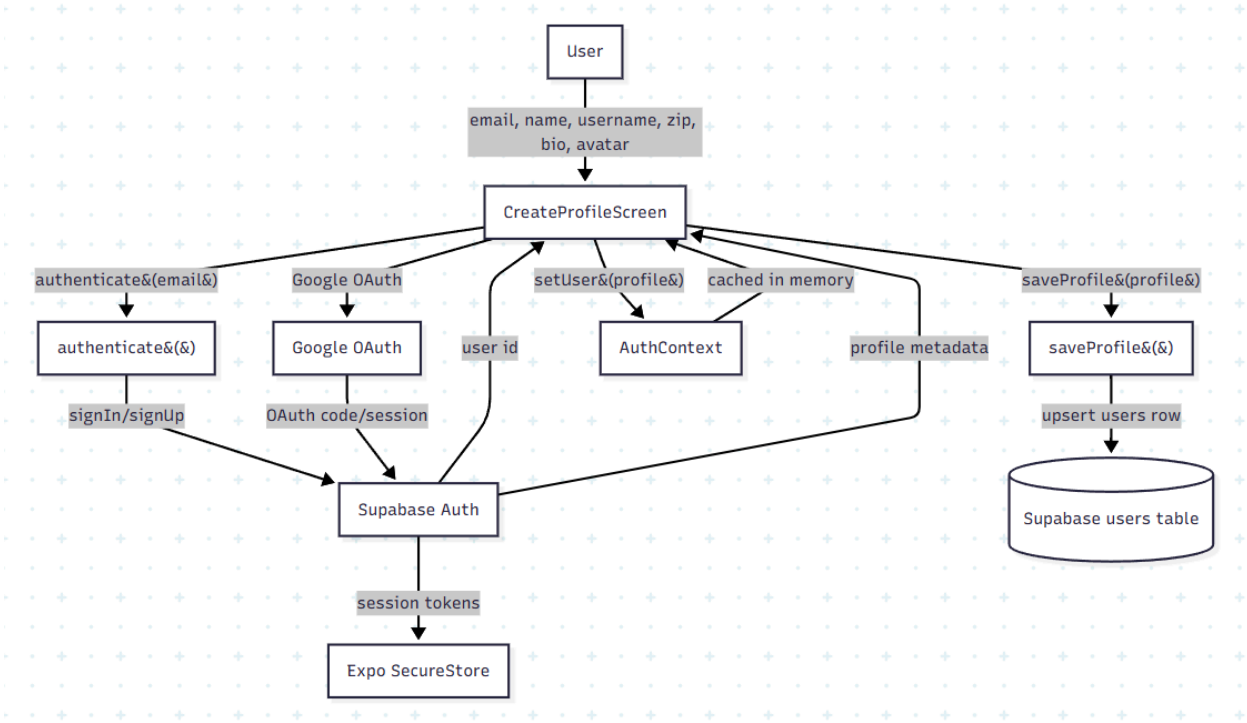
Secondary Owner: Clarence Choy

Date Merged onto the main branch: March 25, 2026

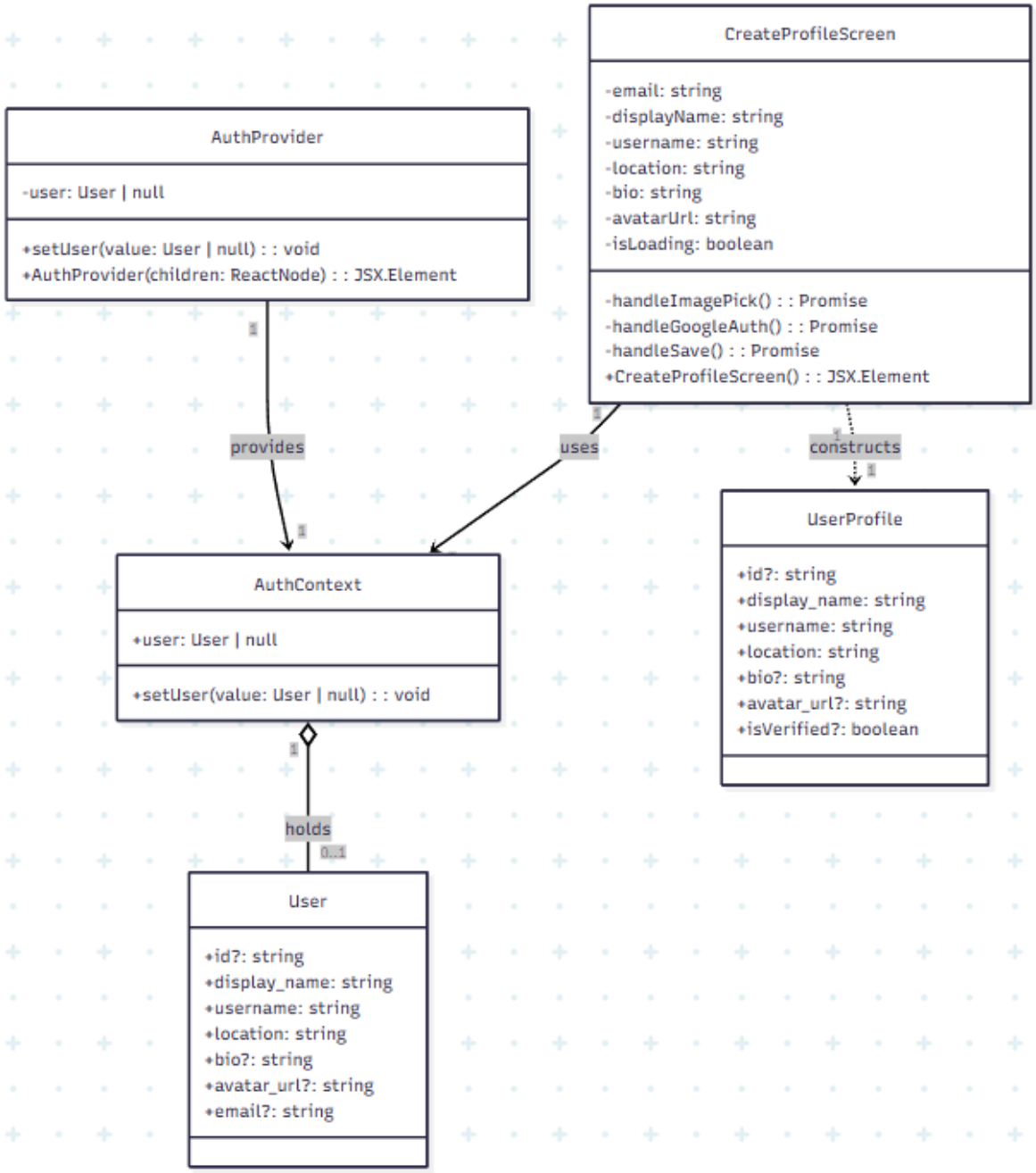
Architecture Diagram



Information Flow:



Class Diagram:



6. Classes, Fields, and Methods (Grouped by Concept)

In this React Native architecture, functional components, contexts, and utility modules represent the "classes."

Concept: UI / Screen Layer

Class: `CreateProfileScreen`

- **Public Fields and Methods:**
  - `CreateProfileScreen()` (Method/Component): The main exported function. Purpose: Renders the profile creation interface and returns the JSX elements to the navigation router.
- **Private Fields and Methods:**
  - `email`, `displayName`, `username`, `location`, `bio`, `avatarUrl`, `isLoading` (Fields): React state variables. Purpose: Temporarily holds the user's form input data in memory and tracks UI loading status before database submission.
  - `handleImagePick()` (Method): Async function. Purpose: Interfaces with the native device file system to allow the user to select an avatar photo.
  - `handleGoogleAuth()` (Method): Async function. Purpose: Initiates the OAuth flow via Supabase and the device's web browser.
  - `handleSave()` (Method): Async function. Purpose: Validates the local form state, authenticates the user, constructs the data payload, and calls the database write utilities.

Concept: Authentication & Session State

Class: `AuthContext` / `AuthProvider`

- **Public Fields and Methods:**
  - `AuthProvider({ children })` (Method/Component): Context provider component. Purpose: Wraps the application tree to make session data globally available.
  - `user` (Field): User object or null. Purpose: Exposes the currently authenticated user's profile data to the rest of the application.
  - `setUser(user)` (Method): State updater function. Purpose: Allows the application to globally update the session state after a successful login or profile edit.
- **Private Fields and Methods:**
  - *Internal State:* The `useState` hook managing the internal `user` data tree before it is exposed to the provider's `value` prop.

Concept: Data Access & API Layer

Class: `SupabaseClient`

- **Public Fields and Methods:**
  - `auth` (Field/Namespace): The authentication API wrapper. Purpose: Exposes methods like `signUp` and `signInWithOAuth` for identity verification.
  - `from(tableName)` (Method): Query builder. Purpose: Initializes database operations (select, insert, upsert) targeting specific PostgreSQL tables.
- **Private Fields and Methods:**
  - *Internal fetch handlers:* (Managed entirely by the library) used to serialize requests and manage the underlying HTTP/WebSocket connections.

7. Technologies, Libraries, and APIs

Technology & Version	Purpose	Rationale	Source / Author / Docs
TypeScript 5.9.2	Core language (typed).	Catches runtime errors at compile time, enforces strict data models, and provides superior IDE intellisense.	Author: Microsoft Docs: typescriptlang.org
React 19.1.0	Declarative UI library.	The industry standard for component-based UI rendering; required dependency for React Native.	Author: Meta Docs: react.dev
React Native 0.81.5	Native mobile framework.	Allows writing a single codebase (TS/JS) that compiles to native iOS and Android UI components.	Author: Meta

			Docs: reactnative.dev
Expo ~54.0.33	Mobile application toolchain.	Abstracts away complex iOS/Android native build environments, provides an extensive suite of device APIs, and enables over-the-air (OTA) updates.	Author: Expo Docs: docs.expo.dev
Expo Router ~6.0.23	File-based routing.	Simplifies app navigation by mirroring the file system hierarchy, reducing boilerplate compared to traditional navigation config objects.	Author: Expo Docs: docs.expo.dev/router
@supabase/supabase-js 2.100.0	Auth & Database Client.	Provides a unified, type-safe API to access hosted PostgreSQL and GoTrue auth, drastically reducing backend setup and maintenance time.	Author: Supabase Docs: supabase.com/docs
expo-secure-store 15.0.8	Encrypted local storage.	Safely stores JWT session tokens using the native iOS Keychain and Android EncryptedSharedPreferences, preventing token theft.	Author: Expo Docs: docs.expo.dev

expo-image-picker 17.0.10	Media selection API.	Provides a standardized, OS-compliant way to request camera roll permissions and extract image URIs for profile avatars.	Author: Expo Docs: docs.expo.dev

8. Database Data Types (Long-Term Storage)

All data is stored in the Supabase PostgreSQL database (`users` table).

- **`id` (Type: `UUID`)**: Links the public profile to the secure auth record. Primary key. **Storage:** 16 bytes.
- **`email` (Type: `TEXT`)**: Used for identity verification, login, and account recovery. **Storage:** ~50-100 bytes (depends on length).
- **`display_name` (Type: `TEXT`)**: The human-readable name shown on the user's profile. **Storage:** ~20-60 bytes.
- **`username` (Type: `TEXT`)**: Unique handle for searching and tagging (e.g., @user). **Storage:** ~10-40 bytes.
- **`location` (Type: `TEXT`)**: Zip code or locality used for geographic discovery/matching. **Storage:** ~10-20 bytes.
- **`bio` (Type: `TEXT`)**: Optional user-provided description of themselves. **Storage:** ~0-500 bytes.
- **`avatar_url` (Type: `TEXT`)**: Pointer URL to the cloud storage bucket containing the image. **Storage:** ~80-150 bytes.

9. Failure Scenarios

Scenario	User-Visible Effect	Internally-Visible Effect
Crashed its process	App instantly closes or freezes to the OS home screen. Upon restart, unsaved form data is gone.	OS kills the process. React memory heap is dumped. AuthContext must be re-initialized from SecureStore.

Lost all runtime state	Form fields abruptly clear. The user may be suddenly kicked back to the login screen.	React state variables reset; user object in memory becomes null.
Erased all stored data	User is forcibly logged out. App behaves as if it's a fresh install.	Local tokens wiped from SecureStore. (If DB is erased: users rows dropped, Auth records deleted).
Database data appears corrupt	UI displays broken images, "Undefined" text, or crashes when attempting to render the profile screen.	Supabase queries return schemas that violate TypeScript expectations, causing rendering logic to fail.
RPC (API call) failed	An error alert pops up (e.g., "Unable to save profile"). The loading spinner stops.	Supabase client throws an error object. The catch block in handleSave() handles the rejection.
Client overloaded	App stutters, UI freezes, button taps are delayed or unresponsive.	Main JS thread is blocked. Frame rate drops. JS event loop is heavily congested.
Client out of RAM	App unexpectedly crashes to the OS home screen.	OS triggers an Out-Of-Memory (OOM) exception and forcefully terminates the application thread.
Database out of space	"Failed to save" error when creating a profile. App remains functional for reading existing data.	Postgres rejects INSERT/UPDATE commands. Backend logs show disk space allocation errors.

Lost network connectivity	Infinite loading spinners, or a "No Internet Connection" toast appears.	Fetch requests fail instantly or time out. SupabaseClient cannot resolve DNS.
Lost access to database	Same as above; user sees generic connection/save errors.	App has internet, but Supabase endpoints return 5xx errors or connection timeouts.
Bot signs up and spams	Users see fake accounts, spam in feeds, or degraded app performance.	Massive spike in users table rows. Postgres compute/bandwidth metrics spike.

---

10. Personally Identifying Information (PII)

List of PII stored and Justification:

- **Email Address:** Needed as the primary unique identifier for secure login, password resets, and critical system notifications.
- **Location (Zip/Locality):** Needed to power the app's location-based features and establish basic community trust.
- **Avatar/Photo:** Needed for community trust, UX personalization, and visual identification.
- **Display Name / Username:** Needed for human-readable app navigation, social searching, and tagging. (Often contains real names).

How it is stored:

- Email is stored in Supabase's isolated, secure auth.users schema (managed by GoTrue).
- Profile metadata (Name, Location, Bio, Avatar) is stored in the public users table within the PostgreSQL database.

Path BEFORE entering long-term storage:

1. **Entry:** User types into text inputs on CreateProfileScreen, or data is pulled from Google during the handleGoogleAuth() flow.
2. **State:** Data enters local React state (email, displayName, username, location, avatarUrl).
3. **Method:** User presses save, triggering handleSave().

4. **Transport:** Data is passed to supabase.auth.signUp() and supabase.from('users').upsert(). It is transmitted over HTTPS to the Supabase backend.

Path AFTER leaving long-term storage:

1. **Read:** App boots and calls supabase.from('users').select().
2. **Context:** Data is injected into AuthContext via the setUser(profile) method.
3. **UI Rendering:** Any UI screen uses the useAuth() hook to read the PII from memory and render it to the screen.

---

11. Security, Auditing, and Minor Policies

Team Responsibilities

- **[Noah Choi]**: Responsible for securing the Supabase PostgreSQL database, writing Row Level Security (RLS) policies, and managing API keys.
- **[Jonathan Gu]**: Responsible for ensuring session tokens are correctly encrypted in device SecureStore and memory is handled safely on the client.
- **[Jonathan Gu]**: Responsible for ensuring the storage and deletion of this data complies with relevant privacy laws (GDPR/CCPA).

Auditing Procedures

- **Routine Access:** Supabase authentication logs and database query logs are retained automatically. Dashboards are reviewed [Insert Frequency, e.g., weekly] for unusual spikes in account creation or read volume. Row Level Security (RLS) ensures the application natively restricts routine access to permitted rows only.
- **Non-Routine Access:** Direct database access (via the Supabase admin dashboard or direct psql connection) is restricted to [Insert Authorized Roles]. Access requires Two-Factor Authentication (2FA) and generates an immutable audit trail logged to [Insert Logging System, e.g., Datadog/AWS CloudTrail].

Minors and Guardian Consent

- **Is PII of a minor under 18 solicited or stored?**
  - No. The platform Terms of Service strictly forbid users under the age of 18. Age-gating during signup prevents the intentional solicitation of minor PII.
- **Does the app solicit guardian permission?**
  - No. Because the application is strictly for adults (18+), COPPA-compliant verifiable guardian consent flows are not implemented.
- **Policy for preventing minor PII access by bad actors:**
  - If a minor bypasses the age gate and their PII enters the system, company policy dictates that the account and all associated PII must be immediately and permanently purged from the database upon discovery. [Insert Moderation Team

Name] handles community reports to identify and ban underage users. Furthermore, since no minor PII is intentionally stored, the specific vector of child abuse suspects accessing valid minor data is mitigated at the system architecture level.